

1. 引言

社区区域气候模式预处理系统（CRESM Preprocessing System，CPS）是为新一代国产区域地球系统模式 CRESM 开发的数据前处理系统，用于统一管理和自动化执行模式运行前所需的各类数据准备任务。CRESM 基于区域气候模式 CWRF，并进一步耦合了新一代通用陆面模式 CoLM2024，通过耦合器架构实现陆—气过程的协同模拟，对数据一致性和前处理流程的规范性提出了更高要求。

在实际应用中，CRESM 的前处理工作涉及多个独立子系统和复杂的数据依赖关系，包括 CWRF 所需的静态地理数据与侧边界驱动场准备、CoLM 所需的基础地表参数场与初始状态场构建，以及 CPL7 耦合器所需的网格映射与插值权重文件生成等步骤。这些流程通常分散在不同工具和脚本中，配置复杂、操作繁琐，且缺乏统一的流程管理机制，容易产生人为配置错误和重复劳动。

CPS 的目标是为 CRESM 提供一套统一、自动化、高性能的数据前处理解决方案。系统通过模块化设计对各前处理环节进行标准化封装，并提供统一的配置接口和流程控制机制，使用户能够以较少的人工干预完成复杂的前处理任务。同时，CPS 支持多平台环境部署，可适配不同计算节点与运行环境，为 CRESM 的批量实验和长期业务化运行提供稳定可靠的技术支撑。

2. 系统环境要求

CPS 系统面向高性能科学计算环境设计，主要运行于 Linux 平台，用于支持 CRESM 模式体系的数据前处理工作。在安装和使用 CPS 之前，请确保计算环境满足以下基本要求。

2.1 操作系统要求

推荐使用基于 Linux 的操作系统环境：

- Linux
 - Ubuntu 20.04 及以上版本(推荐)
- 或其他支持运行CRESM的操作系统

2.2 硬件配置要求

推荐配置（具体需求取决于区域大小和数据规模）：

- CPU：48核及以上
- 内存：100 GB RAM
- 磁盘空间：至少 100 GB 可用空间

2.3 系统软件依赖

CPS 运行过程中依赖多个底层科学计算库和并行计算组件，请确保以下软件已正确安装：

- Intel Fortran 编译器套件
- Intel MPI 并行环境
- NetCDF-C 与 NetCDF-Fortran 库
- HDF5 数据库库
- netCDF Operator (NCO)
- Climate Data Operators (CDO)
- NCAR Command Language (NCL)

建议用户在系统环境中验证以下工具可正常使用：

- `mpirun`
 - `ncdump`
 - `ncl`
-

2.4 Python 运行环境

CPS 基于 Python 运行，建议使用 Conda 进行环境管理，具体环境配置请参考4.1节。
Python 版本要求：

- Python 3.9 及以上版本

主要依赖库包括：

- numpy
 - pandas
 - matplotlib
 - cartopy
 - Pillow
 - geopandas
 - fiona
 - esmpy
 - esmf
 - xesmf
 - pyhdf
 - pyparsing
 - python-dateutil
-

2.5 模式系统依赖

在运行 CPS 之前，需要提前完成以下模式组件的安装与编译配置：

- 区域气候模式 CRESM
- 区域气候模式CWRf的前处理套件(CWPS)

上述组件必须能够独立正常运行，以保证 CPS 前处理流程的正确执行。
我们提供一个标准的工具库CRESM_ToolBox去完成上述系统依赖安装。

2.6 运行环境配置要求

推荐使用单独的环境配置文件（例如：`~/bashrc_cresm`）管理CPS运行环境。

3. 目录结构

CPS 采用模块化目录结构设计，以保证数据、配置文件和程序代码之间的清晰分离，便于系统维护、批量实验管理和跨平台部署。本节对 CPS 的主要目录结构及其功能进行说明。

3.1 项目根目录结构

CPS 项目目录结构如下：

```
1  CPS/
2      |___ PrepScript/           # 核心前处理脚本
3      |___ SpinUpScript/        # CoLM Spin-Up脚本
4      |___ CRESM_ToolBox/       # CRESM 工具箱
5      |___ Case/                # 个例与实验工作目录
6      |___ README.md            # 使用说明文档
```

3.2 核心程序目录（PrepScript）

```
1  PrepScript/
2      |___ CRESM_Preprocessing_System.py # 主入口程序
3      |___ PrepCWRF.py                 # CWRF 前处理模块
4      |___ PrepCoLM.py                 # CoLM 前处理模块
5      |___ PrepCRESM.py                # CPL7 映射文件生成模块
6      |___ env.ini                     # 环境配置文件
7      |___ case.ini                    # 实验配置文件
8      |___ Utils/                      # 内部工具库
9      |___ ProcessScript/              # 外部处理脚本库
10     |___ NML/                        # 模型namelist模板库
11     |___ Forcing/                    # 驱动数据信息库
```

功能说明：

- `CRESM_Preprocessing_System.py`
系统主入口脚本，负责流程调度与模块调用。
- `PrepCWRF.py`
用于执行 CWRF 静态数据处理、IC/BC 生成等流程。
- `PrepCoLM.py`
负责陆面模式 CoLM 的基础数据准备与初始场构建。

- `PrepCRESM.py`
用于生成耦合器所需的网格映射和插值权重文件。

3.3 实验工作目录 (Case)

```
1 Case/
2   |--[GridName]/                # 实验数据汇总目录
3       |--Create_Run_From_CPS.py # 用以创建运行案例的脚本
4       |--Grid_[GridName]/       # 静态数据
5       |--ICBC_[GridName]/       # CWRf初始场及驱动场数据
6   |--PrepCWRf/                  # CWRf相关数据制作目录
7       |--[GridName]/            # CWRf相关数据汇总目录
8       |--First_StaticData/      # 静态数据制作目录
9           |--*Geog_[GridName]/  # CWRf geog数据目录 (对同一区域可复用)
10          |--Second_ICBC/        # CWRf初始场及驱动场数据制作目录
11   |--PrepCoLM/                  # CoLM相关数据制作目录
12       |--[GridName]/            # CoLM相关数据汇总目录
13       |--First_GenMesh/         # CoLM与CWRf网格映射文件制作目录
14       |--Second_MakeSrf/        # CoLM相关静态数据制作目录
15           |--*CoLMSrf_[GridName] # CWRf geog数据目录 (对同一区域和define配置可复用)
16       |--Third_Remap/           # CoLM数据remap至CWRf网格
17   |--PrepCRESM/                 # CRESM相关数据制作目录
18       |--[GridName]/            # cpl7相关数据汇总目录
19   |--NMLS/                      # namelist
20   |--Log/                       # 日志文件目录
```

说明:

- 每个 Case 目录对应一个独立实验区域或分辨率配置
- `Geog_[GridName]` 和 `CoLMSrf_[GridName]` 分别为CWRf和CoLM的静态数据目录, 在同一实验配置下可复用。实验配置由唯一的Geog.ID验证。
- 系统会在该目录下自动生成各模块运行所需的中间文件和结果数据。
- 用户通常只需要管理 Case 层级目录, 无需修改内部自动生成文件。
- **警告:** 如果在实验配置文件中启用了自动清理缓存(CleanTempFiles), 那只有 `Case/[GridName]/` 文件夹会被保留, 请谨慎使用。

4. CPS流程总览

CPS提供了一个集成且自动化的工作流程，该工作流程旨在以统一且可复现的方式处理大气强迫、陆面数据集和耦合器映射文件的预处理。CPS 通过用户定义的配置文件控制的分阶段管道执行预处理任务，总体执行流程如下



5. 安装与部署

5.1 Python 环境部署

CPS 的 Python 运行环境依赖多个科学计算、地理信息处理和数据可视化相关库。为保证依赖版本的一致性与跨平台兼容性，所有 Python 依赖已统一维护在CRESM_ToolBox目录的三个环境配置文件中。为防止环境冲突，我们需要新建3个python环境。

请注意在现有版本中，用户只能通过conda进行环境配置。

用户无需手动逐一安装各个依赖库，只需通过以下方式完成环境配置：

```
1 conda env create -f cresm.yml
2
```

```
3 conda env create -f cresm_xesmf.yml
4
5 conda env create -f Chaomodis.yml
```

以上命令会新建三个新环境：

- **cresm**：负责运行 **main**，**FlowDir**，**plot** 等脚本
- **cresm_xesmf**：负责运行 **GenMesh**，**Remap** 等脚本
- **Chaomodis**：负责运行 **FVC**，**IGBP**，**LAI/SAI**

5.2 CRESM 工具箱安装

CPS提供了一个完整的CRESM工具箱，该工具箱包含区域气候模式CWRF的前处理系统CWPS和依赖数据集，CWRF陆地数据制作相关代码及依赖数据。我们为用户提供了便捷的安装脚本 **SetupAll.py**。用户可以在设置完 **Makeoptions.ini** 之后通过 **SetupAll.py** 一次安装部署所有工具。**Makeoptions.ini** 包含：

```
1 [LIBPATHS]
2 NETCDF_path = $software/netcdf-single-r      # netcdf 安装目录
3 JASPER_path = $software/jasper               # jasper 安装目录
4 PNG_path    = $software/libpng              # libpng 安装目录
5 ZLIB_path   = $software/zlib-1.2.11         # zlib 安装目录
```

配置完上述路径后，您可以通过以下命令完成安装：

```
1 python SetupAll.py
```

6. 配置文件

CRESM Data Preparation System（CPS）通过两个独立但相互关联的配置文件来控制整个前处理流程，分别用于描述具体实验流程（**case.ini**）和运行环境依赖（**env.ini**）。正确理解并配置这两个文件，是系统成功运行的前提。

6.1 系统环境配置文件（**env.ini**）

env.ini 用于描述“在哪里运行 + 用什么运行”，与具体实验无关，env.ini文件包含以下配置：

6.1.1 [Environment] -- System environment information

参数名	类型	说明	示例
SYS_CWRF	str	CWRF 执行所需的环境路径（source 加载）	~/.cresm
SYS_CoLM	str	CoLM 执行所需的环境路径（source 加载）	~/.CoLM_2024
CONDA_CRESM	str	CRESM 前处理所需的 Conda 环境名称	cresm

参数名	类型	说明	示例
CONDA_XESMF	str	4.1节安装的Conda 的 cresm_xesmf 的环境名称	cresm_xesmf
CONDA_CHAO	str	4.1节安装的Conda 的 Chaomodis 的环境名称	Chaomodis
CONDA_UNGRIB	str	4.1节安装的Conda 的 ungrib 的环境名称	ungrib

6.1.2 [Paths] -- System environment information

参数名	类型	说明	示例
ScriptPath	str	PrepScript 文件夹的绝对路径	CPS/PrepScript
CoLMModelPath	str	CoLM 源代码/已编译模型根目录的路径	CRESM/MODEL/CoLM/CoLM202X
CoLMRawDataPath	str	CoLM 原始地理/土壤数据集的路径	/shr03/CoLMrawdata
CoLMRunDataPath	str	CoLM 运行时所需数据集的路径	/shr03/CoLMruntime
CoLMForcingPath	str	CoLM 强迫数据的路径（例如：ERA5-Land）	/shr03/CoLM_Forcing/ERA5LAND/
RootToolBox	str	CRESM 工具箱的根目录	CPS/CRESM_ToolBox
CWPSPath	str	CWRF-CWPS 工具的路径	\${Paths:RootToolBox}/ToolBox/CWRF-CWPS
CWRFToolPath	str	CWRF 工具箱的路径	\${Paths:RootToolBox}/ToolBox/CWRF_Tools
GeogDataPath	str	CWRF Geogrid 二进制数据的路径	geog_wm_modified_lake
LandSeaMaskPath	str	陆地—海洋掩膜数据的路径	land_sea_mask
CWPSStaticPath	str	CWPS 静态表/文件的路径	CWPS_static
GlobalLakeDepth	str	全球湖泊深度数据文件路径	newGlobalLakeDepth.dat
GlobalLakeStatus	str	全球湖泊状态数据文件的路径	newGlobalLakeStatus.dat
WMEJUngrib	str	WMEJ_NC2IM 工具的路径	\${Paths:RootToolBox}/ToolBox/WMEJ_NC2IM
WMEJModis	str	CoLM LAI 数据处理工具的路径	\${Paths:RootToolBox}/ToolBox/WMEJ_modis
ChaoModis	str	MODIS 处理工具的路径	\${Paths:RootToolBox}/ToolBox/Chao_modis
NCOPath	str	ncks 可执行文件的路径	/usr/bin/ncks
CDOPath	str	cdo 可执行文件的路径	/usr/bin/cdo
NCLPath	str	ncl 可执行文件的路径	/usr/bin/ncl

6.1.3 [ForcingName] -- System environment information

参数名	类型	说明	示例
ForcingDataName	str	驱动名称	ERA5
DataCalender	str	驱动数据使用的日历类型（如 standard、noleap）	standard

参数名	类型	说明	示例
Forc_Info	str	驱动信息相关文件（只有NC格式数据需要，GRIB填None）	None
Forc_2D_Path	str	2D平面场文件路径模板（需要完整路径）	SFC<YYYY><MM><DD>.grib
Forc_3D_Path	str	3D高空场文件路径模板（需要完整路径）	PRESS<YYYY><MM><DD>.grib
Forc_SST_Path	str	海表温度场文件路径模板（需要完整路径）	SST<YYYY><MM><DD>.grib



用户可以同时定义多个强迫（例如，[cfsv2]、[gfs]）

6.2 实验配置文件（case.ini）

case.ini 用于控制“做什么 + 在哪里 + 做多久”，决定 CPS 的执行流程与目标实验。

6.2.1 [BaseInfo] -- Global Workflow Settings

参数名	类型	说明	示例
CleanTempFiles	bool	若为 True，则删除中间文件以节省磁盘空间。	True/False
Use_CoLMLAI	bool	若为 True，则使用 CoLM 的 LAI 数据而不是默认的 MODIS 数据制作 LAI。	True/False
Use_CoLMSeaMask	bool	若为 True，则使用 CoLM 的海陆掩膜；若为 False，则使用 shpfile 湖泊/海洋掩膜。	True/False
Enable_TimeChunk	bool	启用时间分块，对较长时间段的 ICBC 处理进行分块并行。	True/False
TimeChunkCount	int	时间分块总数。	4
TimeChunkGroupSize	int	每个串行分组中允许同时处理的最大时间块数量。	2
Reuse_Metgrid	bool	若为 True，仅复用完成标记和全部 met_em 文件均存在的时间块。	True/False
Clean_Ungrib	bool	若为 True，在 Metgrid 和 SST 处理成功后删除 2D、3D、SST Ungrib 中间文件。	True/False

[BaseInfo] 中的所有选项均为必填项。TimeChunkGroupSize 表示每组的时间块数量，各组依次执行，组内时间块并行执行。

当 Reuse_Metgrid = True 时，CPS 会检查时间块目录中的 .metgrid.complete 和全部预期 met_em.d01.*.nc 文件。检查通过后跳过该时间块的 CWPS 文件链接、Ungrib 和 Metgrid，但后续的 Real 步骤仍会执行。

6.2.2 [PrepCWRF] -- CWRF Preprocessing Switches

参数名	类型	说明	示例
CWRFCoreNum	int	用于 CWRF MPI 任务的 CPU 核心数（建议 4-24 个）。	16
Go_ShowDomain	bool	绘制域预览图（检查域覆盖率）。	True/False
Go_Geogrid	bool	运行地理网格，生成静态地理数据。	True/False
Go_FVC	bool	处理植被覆盖度数据。	True/False
Go_LAI	bool	处理叶面积指数数据。	True/False
Go_IGBP	bool	处理 IGBP 土地利用数据。	True/False
Go_SAI	bool	处理茎面积指数数据。	True/False
Collect_GeogData	bool	收集 CWRF 静态数据以便后续复用。	True/False
Go_Ungrib	bool	运行 Ungrib，解码 GRIB/NC 强迫数据。	True/False
Go_Metgrid	bool	运行 Metgrid，将气象数据插值到模型网格。	True/False
Go_Real	bool	运行 Real，生成 <code>wrfinput</code> / <code>wrfbdy</code> 。	True/False
Go_VBS	bool	运行 VBS，进行植被/反照率处理。	True/False
Copy_CWRF_Output	bool	将最终 CWRF 输入复制到案例目录。	True/False

[PrepCWRF] 中的所有选项均为必填项。

6.2.3 [PrepCoLM] -- CoLM Preprocessing Switches

参数名	类型	说明	示例
CoLMCoreNum	int	用于 CoLM MPI 任务的 CPU 核心数（建议 24-128 个）	96
Go_MeshGrid	bool	生成CoLM与CWRF的非结构化网格映射文件	True/False
Go_MakeSrf	bool	生成表面数据集（mksrfdata）	True/False
Go_MakeIni	bool	生成初始条件（mkinidata）	True/False
Go_CoLMTempRun	bool	进行短时运行，生成history	True/False
Go_Remap	bool	重新映射history和restart文件	True/False
Copy_CoLM_Output	bool	将最终的 CoLM 输入复制到案例目录 （GridName/PrepCoLM/GridName）	True/False

6.2.4 [PrepCRESM] -- Coupler weight file

参数名	类型	说明	示例
Go_Coupler_Prep	bool	为耦合器生成权重和映射	True/False

[PrepCRESM] 中的所有选项均为必填项。

6.2.5 [GatherData] -- Final Output Collection

参数名	类型	说明	示例
Collect_CWRF_Output	bool	收集最终的CWRF输入数据	True/False
Collect_CoLM_Output	bool	收集最终的CoLM输入数据	True/False
Collect_CRESM_Output	bool	收集耦合器名称列表和映射文件（GridName/GridName）	True/False

[GatherData] 中的所有选项均为必填项。

6.2.6 [GridName] -- Domain & Experiment Specifics

每个实验 [GridName] 小节中，除 LakeThreshold 外的所有选项均为必填项。

LakeThreshold 可以省略或留空，系统将默认使用 0.5；如果填写，则取值范围必须为 0.0-1.0。

参数名	类型	说明	示例
CaseOutputPath	str	案例输出文件的根目录	/path/to/CPS/Case
ForcingDataName	str	驱动名称，必须与 env.ini 中的 forcing 小节对应（如 CFSV2、ERA5、MPI-ESM1-2-HR_ssp245）	ERA5
StartTime	str	开始时间，格式为 YYYY-MM-DD_HH:MM:SS	2000-01-01_00:00:00
EndTime	str	结束时间，格式为 YYYY-MM-DD_HH:MM:SS	2000-01-03_00:00:00
EdgeNum_WE	int	东西向网格数+1	232
EdgeNum_SN	int	南北向网格数+1	172
dx_WE	float	东西向网格分辨率 (单位: m)	30000
dy_SN	float	南北向网格分辨率 (单位: m)	30000
RefLat	float	该区域的中心纬度	35.1778
RefLon	float	该区域的中心经度	110.0
True_Lat1	float	兰伯特投影的下切线纬度	30.0
True_Lat2	float	兰伯特投影的上切线纬度	60.0
StandLon	float	兰伯特投影的标准经度	110.0
BdyWidth	int	侧向边界松弛宽度，必须为不小于 13 的奇数。	15
LakeThreshold	float	可选湖泊比例（0.0-1.0），湖泊比例大于此值的网格单元将被视为湖泊点；缺失或留空时默认为 0.5。	0.5
MeshSize	int	CoLM 与 CWRF 的非结构化网格映射文件分辨率，取值为 1、2 或 3。1=Coarse, 2=Medium, 3=Fine。	1



用户可以同时定义多个实验配置（例如，[CN_15km]、[Yangtze]）

上述 `case.ini` 的 Section 和选项检查均由 `CRESM_Preprocessing_System.py` 集中完成，后续处理模块使用已经完成检查的配置，不再重复进行配置检查。

7. 基本用法

7.1 使用前准备

在运行 CRESM 数据前处理系统之前，请确保以下准备工作已完成：

1. 系统环境已正确配置
 - 已完成 `env.ini` 中所有路径与环境的配置；
 - 相关系统环境（CWRF、CoLM）与 Conda 环境均可正常激活；
 - 所有外部程序（如 `ncl`、`cdo`、`ncks` 等）可在命令行直接调用。
2. 实验配置文件已准备
 - 针对具体实验案例，正确填写 `case.ini`；
 - 确保 `ForcingDataName` 与 `env.ini` 中对应的 `forcing` 配置一致；
 - 检查模拟时间、网格参数、分辨率等关键信息。

7.1 命令行参数说明

CRESM 数据前处理系统通过统一的主程序启动，提供了诸多选项，用户可以是同 `-h` 选项查看命令行帮助

命令

```
1 | python CRESM_Preprocessing_System.py -h
```

输出(示例)

```
1 | usage: CRESM_Preprocessing_System.py [-h] [-v] [-d] [-ch] [-l] [-n NAME] [-g DIR]
2 |                                     [-s DIR] [-y YEAR] [-c NAME]
3 |
4 |
5 | CRESM Preprocessing System (CPS)
6 |
7 | optional arguments:
8 |
9 |   -h, --help            show this help message and exit
10 |   -v, --version         Show version information and exit
11 |   -d, --debug           Enable debug mode
12 |   -ch, --confighelp     Display configuration help and exit
13 |   -l, --listcases       List available "useful cases" in case.ini and exit
14 |   -n NAME, --gridname NAME
15 |                         Specify the case name (matches a section in case.ini)
16 |   -g DIR, --geogdir DIR
17 |                         Use existing geography data from DIR for IC/BC preparation;
18 |                         if omitted, new geography data will be generated
```

```
19 -s DIR, --colmsrf DIR
20           Use existing CoLM surface data from DIR for PrepCoLM preparation;
21           if omitted, new surface data will be generated
22 -y YEAR, --year YEAR Override start/end year in the case (for yearly forcing data),
23           e.g., --year 2023
24 -c NAME, --collectcase NAME
25           When using --year, gather all annual forcing data into the folder NAME;
26           inside that folder, each forcing file is renamed with the year suffix.
```

7.1.1 -v, --version

显示程序版本信息并退出。

- 用于确认当前使用的 CPS 版本
 - 便于问题反馈与版本对齐
-

7.1.2 -d, --debug

启用调试模式（**Debug Mode**）。

- 打印更详细的日志信息
 - 包括路径检查、环境检查、每一步调用的外部命令
 - 建议在出现错误时开启
-

7.1.3 -ch, --confighelp

显示配置文件说明并退出。

- 打印 `case.ini` 和 `env.ini` 的完整配置说明
 - 用于快速查阅配置含义，无需打开手册
-

7.1.4 -l, --listcases

列出 `case.ini` 中已定义的可用实验案例并退出。

- 自动扫描 `case.ini`
 - 显示所有可直接运行的 [GridName] 小节
 - 不执行任何计算流程
-

7.1.5 -n NAME, --gridname NAME

指定要运行的实验案例名称。

- NAME 必须与 case.ini 中的某个 [GridName] 小节一致
- 这是实际运行 CPS 时的核心参数
-

7.1.6 -g DIR, --geogdir DIR

使用已有的 CWRP 地理静态数据目录。

若指定该参数：

- 跳过 Geogrid/FVC/LAI/SAI 等静态地理数据生成步骤
- 直接使用 DIR 中已有的数据
- 网格配置有 Geog_[GridName] 中的 Geog.ID 文件确认。不同网格配置不能混用。

Geog_ID样式：

```
1 X[232]_Y[172]_dx[30000.000000]_dy[30000.000000]_lat[35.177800]_lon[110.000000]_
2 t11[30.000000]_t12[60.000000]_slon[110.000000]_bw[15]_lk[0.500000]
```

7.1.7 -s DIR, --colmsrf DIR

使用已有的 CoLM 地表数据目录。

若指定该参数：

- 跳过 PrepCoLM 中的地表数据生成流程 MakeSrf
- 网格配置有 CoLMSrf_[GridName] 中的 Geog.ID 文件确认。不同网格配置不能混用。
- 多案例共享同一套 CoLM surface 数据建议使用

7.1.8 -y YEAR, --year YEAR

覆盖 case.ini 中定义的起止年份，仅运行指定年份，这会自动生成新实验 [GridName].year。

- YEAR 为四位整数（如 2023）
- 自动调整：
 - StartTime (如：2023-12-01_00:00:00)
 - EndTime (如：2025-01-05_00:00:00)
 - 起止时间为 year-1年12月1日至 year+1年1月5日
- 主要用于逐年强迫数据处理, 或跨年运行。

7.1.9 -c NAME, --collectcase NAME

年度模式下的数据汇总参数（与 --year 联合使用）。

- 将多个年度的强迫数据汇总到同一目录 NAME
 - 每个文件自动追加年份后缀
 - 便于后续多年模拟或批处理
- 注意：仅在使用 `--year` 后单独使用才有效

7.2 运行

7.2.1 最小运行示例

当第一次运行一个新案例时推荐使用以下指令运行：

```
1 | python CRESM_Preprocessing_System.py -n CN_30km
```

7.2.2 制作多年数据

当需要制作多年数据时，推荐先只运行5天，生成静态数据 `CoLMSrf_[GridName]` 和 `Geog_[GridName]`。

```
1 | python CRESM_Preprocessing_System.py -n CN_30km
```

之后复用静态数据再生成多年数据 (以2000年为例)

```
1 | python CRESM_Preprocessing_System.py -n CN_30km -y 2000 \  
2 |                                     -g path/to/Geog_[GridName] \  
3 |                                     -s path/to/CoLMSrf_[GridName]
```

再所有年份都运行完毕后，在使用收集模式对所有年度数据进行收集

```
1 | python CRESM_Preprocessing_System.py -c CN_30km
```

7.2.3 调试运行

由于CPS是一个十分复杂的系统，因此当出错时可能需要使用调试模式以确定出错的具体环节。建议当出错时仅开启当前流程(`Go_`)，并启用 `-d` 模式

```
1 | python CRESM_Preprocessing_System.py -n CN_30km -d
```

8. 创建运行案例

通过CPS制作的数据既可以使用以前的 `create_run.py` 创建运行案例，也可以使用更方便的 `Create_Run_From_CPS.py` 创建运行案例。`Create_Run_From_CPS.py` 有以下可选参数：

```
1  Create run directory for CRESM
2
3  optional arguments:
4  -h, --help            show this help message and exit
5  -casename CASENAME, --casename CASENAME
6                        name for entire run
7  -casedir CASEDIR, --casedir CASEDIR
8                        The parent directory where the case will be created
9  -griddir GRIDDIR, --griddir GRIDDIR
10                       Full path to Grid directory
11  -icbcdir ICBCDIR, --icbcdir ICBCDIR
12                       Full path to ICBC directory
13  -st STARTTIME, --starttime STARTTIME
14                       YYYYMMDDHH
15  -et ENDTIME, --endtime ENDTIME
16                       YYYYMMDDHH
17  -ncpus NCPUS, --ncpus NCPUS
18                       number of cpus
19  -jms {lsf,slurm}, --job_management_system {lsf,slurm}
20  -ini INI, --ini INI   config.ini path
21  -yearly, --yearly     If true, link ICBC files without renaming
22                       and copy wrfinput for start year
23  -fvcom FVCOM, --fvcom FVCOM
24                       True/False
```

其中，`casename` 是案例名称，`casedir` 是案例创建目录，`griddir` 是静态数据目录(即 `Grid_[GridName]`)。`icbcdir` 是CWRP初始化和测边界场数据。`starttime` 和 `endtime` 分别是开始时间和结束时间。`ncpus` 是使用的核心数。`job_management_system` 是任务管理系统，目前仅支持 `lsf`，`slurm`，`ini` 是 `prep.ini` 文件的路径。`yearly` 用以区分是否是分年的ICBC。如果数据是分年(即有年份后缀)，但是未启用，则只链接第一年的数据。

9. 日志

CRESM Preprocessing System (CPS) 采用分级、分流程的日志机制，以便用户在复杂的数据前处理过程中进行状态跟踪、问题诊断和结果回溯。

日志系统主要由主日志（**Main Log**）和流程日志（**Process Logs**）两部分组成。

9.1 主日志（Main Log）

日志位置

主日志文件默认保存在当前运行目录，文件命名格式为：

```
1 | DataPrepare.[GridName].log
```

其中 `GridName` 对应命令行参数 `--gridname` 指定的案例名称。

主日志用于记录整个 CPS 工作流的全局信息，主要包括：

- 程序启动与结束信息
- 配置文件读取与参数解析结果
- 系统环境与路径检查结果
- 各预处理模块的开始与结束状态
- 外部程序调用命令（Debug 模式下）
- 全局错误与异常回溯信息

日志级别：

- `INFO`：关键流程节点与总体状态信息
- `DEBUG`：详细执行信息（仅在启用 `--debug` 时输出）
- `WARNING`：非致命问题提示
- `ERROR`：流程中断或执行失败信息

建议：

- 日常生产运行可关闭 Debug
- 初次部署或出错时开启 `--debug`

9.2 流程日志（Process Logs）

日志位置

主日志文件默认保存在当前运行目录，文件命名格式为：

```
1 | CaseOutputPath/[GridName]/Log/
```

该目录用于集中存放各个处理步骤的标准输出与错误输出。

命名规则

流程日志采用如下统一命名格式：

```
1 | log.<ProcessName>[.<TimeTag>]
```

在涉及时间分片或多次调用的流程中，日志文件可能附带时间标识或编号后缀。

日志内容

流程日志主要记录：

- 外部可执行程序（如 `geogrid.exe`，`ungrib.exe`）的标准输出
- MPI 并行运行信息
- 模型或工具自身产生的错误信息

- 详细的数值或文件处理过程
这些日志通常是定位失败原因的第一手信息。

9.3 日志与错误定位建议

当 CPS 执行失败时，建议按以下顺序进行排查：

1. 查看主日志
 - 确认失败发生的流程阶段及路径（会提示 **Error at:**）
 - 检查环境、路径或参数相关错误
2. 查看对应流程日志
 - 打开 `CaseOutputPath/[GridName]/Log/Log.<ProcessName>`
 - 检查是否存在：
 - 库缺失（如 `libnetcdf*.so`）
 - 输入数据不存在
 - MPI 或并行错误
3. 开启 Debug 模式重试

Danger

若启用了 `CleanTempFiles = True`，部分中间文件可能被自动删除，因此 Debug 时请勿开启该模式

9.4 常见错误排查建议

在目前版本中，部分操作系统可能会出现以下错误：

1. 无法找到依赖库，如 `netcdf`

- 请确保在运行 CPS 前，source 对应环境后重试，目前在 CPS 内部切换环境的功能并不稳定

2. CWRF 制作 ICBC 数据时出错且原因未知

- 部分网格的划分数会与分配的核心数冲突，因此可以尝试更改分配的核心数

3. CoLM 部分允许失败且原因未知

- 可能时由于环境加载有误，请重新加载环境再次允许

Caution

请注意，CoLM 的 `define` 选项会影响你的目标实验配置，请确保 `define` 正确